Q1)

1. Design:

```
module adder (
    input  clk,
    input  reset,
    input  signed [3:0] A, // Input data A in 2's complement
    input  signed [3:0] B, // Input data B in 2's complement
    output reg signed [4:0] C // Adder output in 2's complement
        );

   // Register output C
   always @(posedge clk or posedge reset) begin
      if (reset)
        C <= 5'b0;
      else
        C <= A + B;
   end
endmodule
```

2. Verification Plan:

Label	Design Requirment Description	Stimulus Generation	Funcational Coverage	Funcational Checking
ADDER 1	When the reset is asserted, the output C value must be low	Directed at the start of the simulation		A checker in the testbench to make sure the output is correct
ADDER_2	Verifying maximum negative value on A and maximum negative value on B	Directed during the simulation		A checker in the testbench to make sure the output is correct
ADDER_3	Verifying maximum negative value on A and zero value on B	Directed during the simulation		A checker in the testbench to make sure the output is correct
ADDER_4	Verifying maximum negative value on A and maximum positive value on B	Directed during the simulation		A checker in the testbench to make sure the output is correct
ADDER_5	Verifying maximum positive value on A and maximum negative value on B	Directed during the simulation		A checker in the testbench to make sure the output is correct
ADDER_6	Verifying maximum positive value on A and zero value on B	Directed during the simulation		A checker in the testbench to make sure the output is correct
ADDER_7	Verifying maximum positive value on A and maximum positive value on B	Directed during the simulation		A checker in the testbench to make sure the output is correct
ADDER_8	Verifying zero on A and maximum Negative value on B	Directed during the simulation		A checker in the testbench to make sure the output is correct
ADDER_9	Verifying zero on A and zero value on B	Directed during the simulation		A checker in the testbench to make sure the output is correct
ADDER_10	Verifying zero on A and maximum positive value on B	Directed during the simulation		A checker in the testbench to make sure the output is correct
ADDER_11	When the reset is asserted, the output C value must be low	Directed at the end of the simulation		A checker in the testbench to make sure the output is correct

3. Testbench Code:

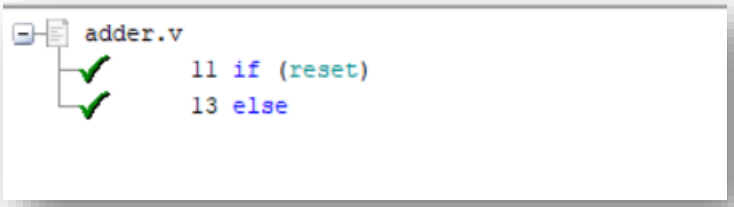
```
module adder_tb();

    localparam MAXPOS = 7;
    localparam MAXNEG = -8;

    logic  CLK;
    logic  reset;
    logic  signed [3:0] A;  // Input data A in 2's complement
    logic  signed [3:0] B;  // Input data B in 2's complement
    logic  signed [4:0] C_dut; // Adder output in 2's complement

    int error_count, correct_count;

    adder DUT (CLK, reset, A, B, C_dut);

    initial begin
       CLK = 0;
       forever
           #1 CLK = ~CLK;
   end

   task reset_assert();
    reset = 1;
    check_result(0);
    reset = 0;
   endtask

    task check_result(input logic signed [4:0] C_expected);
        @(negedge CLK);
        $display("Expected:%d | DUT: %d", C_expected, C_dut);
        if(C_dut !== C_expected) begin
            error_count++;
            $stop;
        end
        else begin
            correct_count++;
        end
    endtask

    initial begin
        A = 0;
        B = 0;
        correct_count = 0;
        error_count = 0;

        //Adder_1
        reset_assert();

        //Adder_2
        A = MAXNEG; B = MAXNEG; check_result(-16);
        //Adder_3
        A = MAXNEG; B = 0; check_result(MAXNEG);
        //Adder_4
        A = MAXNEG; B = MAXPOS; check_result(-1);

        //Adder_5
        A = MAXPOS; B = MAXNEG; check_result(-1);
        //Adder_6
        A = MAXPOS; B = 0; check_result(MAXPOS);
        //Adder_7
        A = MAXPOS; B = MAXPOS; check_result(14);

        //Adder_8
        A = 0; B = MAXNEG; check_result(MAXNEG);
        //Adder_9
        A = 0; B = 0; check_result(0);
        //Adder_10
        A = 0; B = MAXPOS; check_result(MAXPOS);

        //Adder_11
        reset_assert();

        $display("Test Finished | Correct_Count =%d | Error_Count=%d", correct_count, error_count);
        $stop;

    end

endmodule
```

4. Do File:

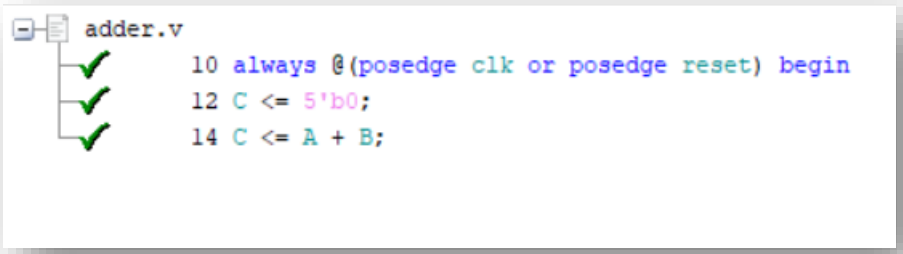
```
vlib work
vlog adder.v adder_tb.sv  +cover -covercells
vsim -voptargs=+acc work.adder_tb -cover
add wave *
coverage save adder_tb.ucdb -onexit -du work.adder
run -all
quit -sim
vcover report adder_tb.ucdb -details -annotate -all -output coverage_rpt.txt
```

5. Code Coverage:

Branch Coverage:

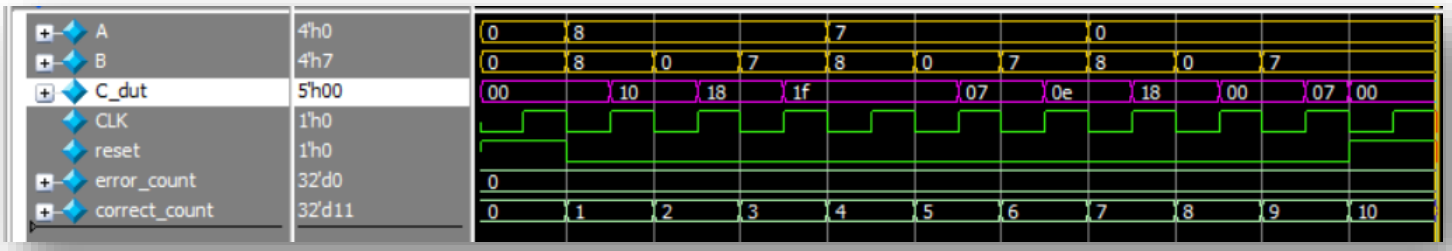


Statement Coverage:



Toggle Coverage:

6. Questasim Waveform:



7. Coverage Report:

Branch Coverage:

```
================================================================================
=== Instance: /\adder_tb#DUT
=== Design Unit: work.adder
================================================================================
Branch Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Branches                         2         2         0   100.00%

================================Branch Details================================

Branch Coverage for instance /\adder_tb#DUT

    Line         Item                      Count     Source
    ----         ----                      -----     ------
  File adder.v
------------------------------------IF Branch------------------------------------
    11                                        13     Count coming in to IF
    11              1                          4           if (reset)

    13              1                          9           else

Branch totals: 2 hits of 2 branches = 100.00%
```

Statement Coverage:

```
Statement Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Statements                       3         3         0   100.00%

================================Statement Details================================
```

Toggle & Total Coverage:

```
================================Toggle Details================================

Toggle Coverage for instance /\adder_tb#DUT  --

                                          Node      1H->0L      0L->1H  "Coverage"
                                          ----------------------------------------
                                        A[0-3]           1           1      100.00
                                        B[0-3]           1           1      100.00
                                        C[4-0]           1           1      100.00
                                           clk           1           1      100.00
                                         reset           1           1      100.00

Total Node Count     =         15
Toggled Node Count   =         15
Untoggled Node Count =          0

Toggle Coverage      =     100.00% (30 of 30 bins)


Total Coverage By Instance (filtered view): 100.00%
```

Q2)

1. Design:

```
1  module priority_enc (
2  input  clk,
3  input  rst,
4  input  [3:0] D,
5  output reg [1:0] Y, //Fixed bug: Added Reg modifier
6  output reg valid
7  );
8
9  always @(posedge clk) begin
10   if (rst) begin
11      Y <= 2'b0;
12      valid <= 1'b0;          //Fixed bug: Added Missing line as rst resets all outputs
13   end
14   else begin                 //Fixed bug: added Begin and end to Else statement
15     casex (D)
16         4'b1000: Y <= 0;
17         4'bX100: Y <= 1;
18         4'bXX10: Y <= 2;
19         4'bXXX1: Y <= 3;
20     endcase
21     valid <= (~|D)? 1'b0: 1'b1;
22   end
23 end
24 endmodule
```

2. Verification Plan:

Label	Design Requirment Description	Stimulus Generation	Funcational Coverage	Funcational Checking
Priorirt_Encoder_1	When the reset is asserted, the output Y and Valid value must be low	Directed during the simulation		A checker in the testbench to make sure the output is correct
Priorirt_Encoder_2	exhaustively verify all possible of input D combination.	Directed during the simulation		A checker in the testbench to make sure the output is correct

3. Testbench Code:

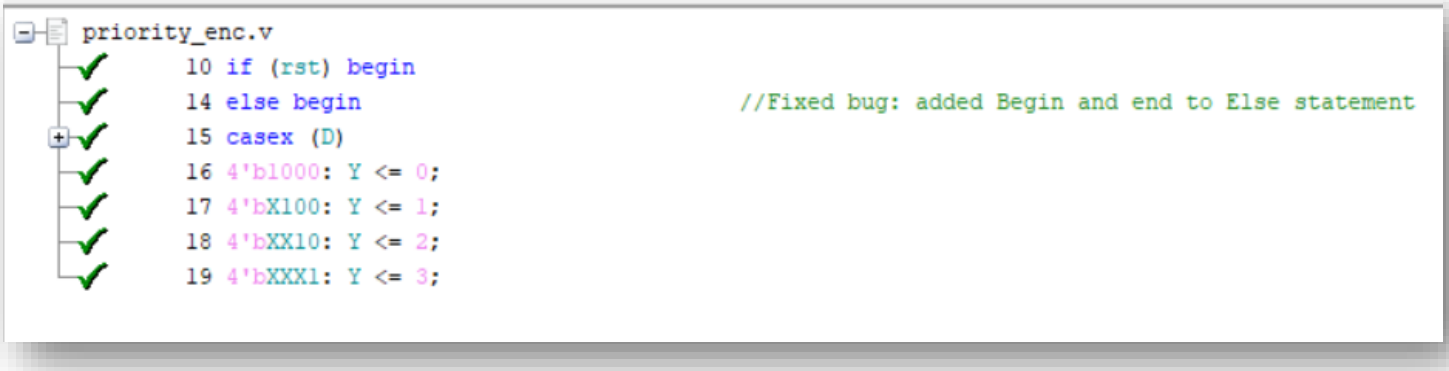
```
module priority_enc_tb();

logic  clk;
logic  rst;
logic  [3:0] D;
logic  [1:0] Y_dut;
logic  valid_dut;

int error_count, correct_count;

priority_enc dut (clk, rst, D, Y_dut, valid_dut);

initial begin
    clk = 0;
    forever
        #1 clk = ~clk;
end

task check_result(input logic valid_expected, [1:0] Y_expected);
    @(negedge clk);
    $display("Y: Expected:%d | DUT:%d || Valid: Expected:%d | DUT:%d", Y_expected, Y_dut, valid_expected, valid_dut);
    if(Y_dut !== Y_expected || valid_dut !== valid_expected)begin
        error_count++;
        $stop;
    end
    else begin
        correct_count++;
    end
endtask

task reset_assert();
    rst = 1;
    check_result(0, 0);
    rst = 0;
endtask

initial begin

    correct_count = 0;
    error_count = 0;

    //Priority_Encoder_1
    reset_assert();

    //Priority_Encoder_2
    D = 4'b0000; check_result(0, 0); //Y_expected is Don't Care
    D = 4'b1000; check_result(1, 0);

    D = 4'b0100; check_result(1, 1);
    D = 4'b1100; check_result(1, 1);

    D = 4'b0010; check_result(1, 2);
    D = 4'b0110; check_result(1, 2);
    D = 4'b1010; check_result(1, 2);
    D = 4'b1110; check_result(1, 2);

    D = 4'b0001; check_result(1, 3);
    D = 4'b0011; check_result(1, 3);
    D = 4'b0101; check_result(1, 3);
    D = 4'b0111; check_result(1, 3);
    D = 4'b1001; check_result(1, 3);
    D = 4'b1011; check_result(1, 3);
    D = 4'b1101; check_result(1, 3);
    D = 4'b1111; check_result(1, 3);

    D = 4'b1000; check_result(1, 0); //For 100% Code Coverage

    //Priority_Encoder_1
    reset_assert();

    $display("Total Tests:%d | Correct Tests:%d | Error Test:%d", correct_count + error_count, correct_count, error_count);
    $stop;

end

endmodule
```

4. Do File:

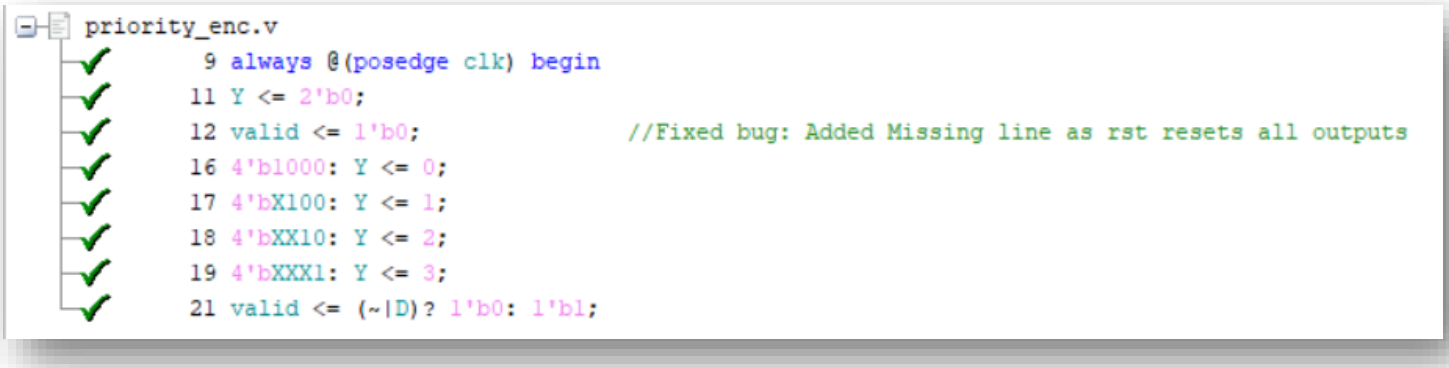
```
vlib work
vlog priority_enc.v priority_enc_tb.sv  +cover -covercells
vsim -voptargs=+acc work.priority_enc_tb -cover
add wave *
coverage save priority_enc.ucdb -onexit -du work.priority_enc
run -all
#quit -sim
#vcover report priority_enc.ucdb -details -annotate -all -output coverage_rpt.txt
```

5. Code Coverage:

Branch Coverage:

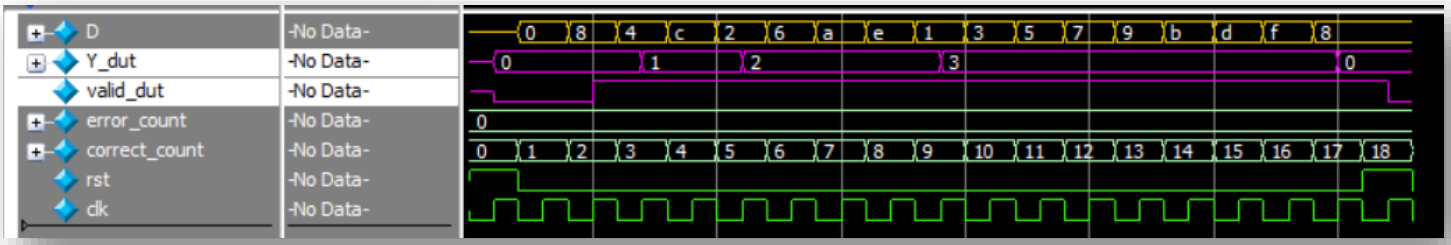


Statement Coverage:



Toggle Coverage:

6. Questasim Waveform:



7. Coverage Report:

Branch Coverage:

```
Branch Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Branches                         7         7         0   100.00%

================================Branch Details================================

Branch Coverage for instance /\priority_enc_tb#dut

    Line         Item                      Count     Source
    ----         ----                      -----     ------
  File priority_enc.v
-----------------------------------IF Branch-----------------------------------
    10                                        19     Count coming in to IF
    10              1                          2       if (rst) begin

    14              1                         17       else begin

Branch totals: 2 hits of 2 branches = 100.00%

---------------------------------CASE Branch---------------------------------
    15                                        17     Count coming in to CASE
    16              1                          2               4'b1000: Y <= 0;

    17              1                          2               4'bX100: Y <= 1;

    18              1                          4               4'bXX10: Y <= 2;

    19              1                          8               4'bXXX1: Y <= 3;

                                               1     All False Count
Branch totals: 5 hits of 5 branches = 100.00%
```

Statement Coverage:

```
Statement Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Statements                       8         8         0   100.00%
```

Toggle & Total Coverage:

```
Toggle Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Toggles                         18        18         0   100.00%

================================Toggle Details================================

Toggle Coverage for instance /\priority_enc_tb#dut  --

                                          Node      1H->0L      0L->1H  "Coverage"
                                          ----------------------------------------
                                       D[0-3]           1           1      100.00
                                       Y[1-0]           1           1      100.00
                                          clk           1           1      100.00
                                          rst           1           1      100.00
                                        valid           1           1      100.00

Total Node Count     =          9
Toggled Node Count   =          9
Untoggled Node Count =          0

Toggle Coverage      =     100.00% (18 of 18 bins)


Total Coverage By Instance (filtered view): 100.00%
```

Q3)

1. Design:

```
module priority_enc (
input  clk,
input  rst,
input  [3:0] D,
output reg [1:0] Y, //Fixed bug: Added Reg modifier
output reg valid
);

always @(posedge clk) begin
  if (rst) begin
     Y <= 2'b0;
     valid <= 1'b0;          //Fixed bug: Added Missing line as rst resets all outputs
  end
  else begin                 //Fixed bug: added Begin and end to Else statement
    casex (D)
        4'b1000: Y <= 0;
        4'bX100: Y <= 1;
        4'bXX10: Y <= 2;
        4'bXXX1: Y <= 3;
    endcase
    valid <= (~|D)? 1'b0: 1'b1;
  end
end
endmodule
```

2. Verification Plan:

Label	Design Requirment Description	Stimulus Generation	Funcational Coverage	Funcational Checking
ALU_1	When the reset is asserted, the output C value must be low	Directed during the start of simulation		A checker in the testbench to make sure the output is correct
ALU_2	Verifying Addition of maximum positive value on A and maximum positive value on B	Directed during the simulation		A checker in the testbench to make sure the output is correct
ALU_3	Verifying Addition of maximum postive value on A and maximum Negative value on B	Directed during the simulation		A checker in the testbench to make sure the output is correct
ALU_4	Verifying Addition of maximum negative value on A and maximum positive value on B	Directed during the simulation		A checker in the testbench to make sure the output is correct
ALU_5	Verifying Addition of maximum negative value on A and maximum negative value on B	Directed during the simulation		A checker in the testbench to make sure the output is correct
ALU_6	Verifying Subtraction of maximum positive value on A and maximum positive value on B	Directed during the simulation		A checker in the testbench to make sure the output is correct
ALU_7	Verifying Subtraction of maximum postive value on A and maximum Negative value on B	Directed during the simulation		A checker in the testbench to make sure the output is correct
ALU_8	Verifying Subtraction of maximum negative value on A and maximum positive value on B	Directed during the simulation		A checker in the testbench to make sure the output is correct
ALU_9	Verifying Subtraction of maximum negative value on A and maximum negative value on B	Directed during the simulation		A checker in the testbench to make sure the output is correct
ALU_10	Verifying bitwise invert of maximum value on A	Directed during the simulation		A checker in the testbench to make sure the output is correct
ALU_11	Verifying reduction OR on B by applying a traveling bit on its bits like one hot	Directed during the simulation		A checker in the testbench to make sure the output is correct

3. Testbench Code:

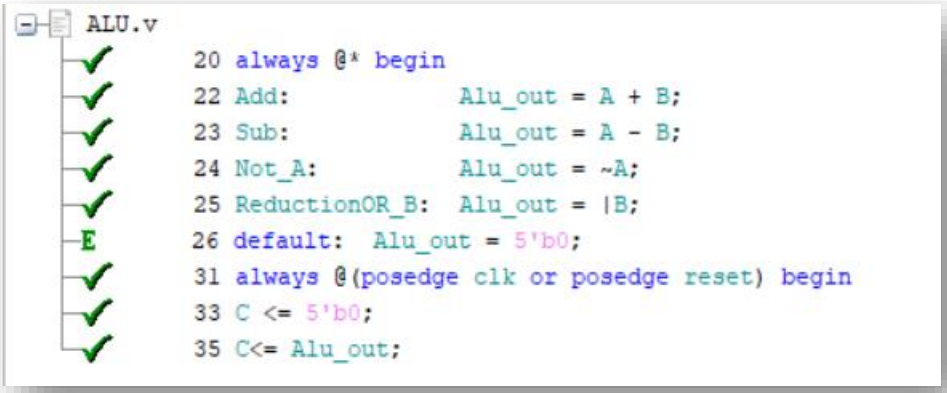
```
module priority_enc_tb();

logic  clk;
logic  rst;
logic  [3:0] D;
logic  [1:0] Y_dut;
logic  valid_dut;

int error_count, correct_count;

priority_enc dut (clk, rst, D, Y_dut, valid_dut);

initial begin
    clk = 0;
    forever
        #1 clk = ~clk;
end

task check_result(input logic valid_expected, [1:0] Y_expected);
    @(negedge clk);
    $display("Y: Expected:%d | DUT:%d || Valid: Expected:%d | DUT:%d", Y_expected, Y_dut, valid_expected, valid_dut);
    if(Y_dut !== Y_expected || valid_dut !== valid_expected)begin
        error_count++;
        $stop;
    end
    else begin
        correct_count++;
    end
endtask

task reset_assert();
    rst = 1;
    check_result(0, 0);
    rst = 0;
endtask

initial begin

    correct_count = 0;
    error_count = 0;

    //Priority_Encoder_1
    reset_assert();

    //Priority_Encoder_2
    D = 4'b0000; check_result(0, 0); //Y_expected is Don't Care
    D = 4'b1000; check_result(1, 0);

    D = 4'b0100; check_result(1, 1);
    D = 4'b1100; check_result(1, 1);

    D = 4'b0010; check_result(1, 2);
    D = 4'b0110; check_result(1, 2);
    D = 4'b1010; check_result(1, 2);
    D = 4'b1110; check_result(1, 2);

    D = 4'b0001; check_result(1, 3);
    D = 4'b0011; check_result(1, 3);
    D = 4'b0101; check_result(1, 3);
    D = 4'b0111; check_result(1, 3);
    D = 4'b1001; check_result(1, 3);
    D = 4'b1011; check_result(1, 3);
    D = 4'b1101; check_result(1, 3);
    D = 4'b1111; check_result(1, 3);

    D = 4'b1000; check_result(1, 0); //For 100% Code Coverage

    //Priority_Encoder_1
    reset_assert();

    $display("Total Tests:%d | Correct Tests:%d | Error Test:%d", correct_count + error_count, correct_count, error_count);
    $stop;

end

endmodule
```

4. Do File:

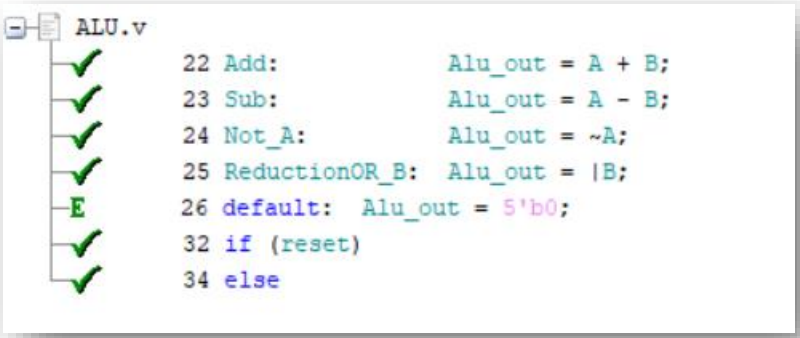
```
vlib work
vlog priority_enc.v priority_enc_tb.sv  +cover -covercells
vsim -voptargs=+acc work.priority_enc_tb -cover
add wave *
coverage save priority_enc.ucdb -onexit -du work.priority_enc
run -all
#quit -sim
#vcover report priority_enc.ucdb -details -annotate -all -output coverage_rpt.txt
```

5. Code Coverage:

Statment Coverage:

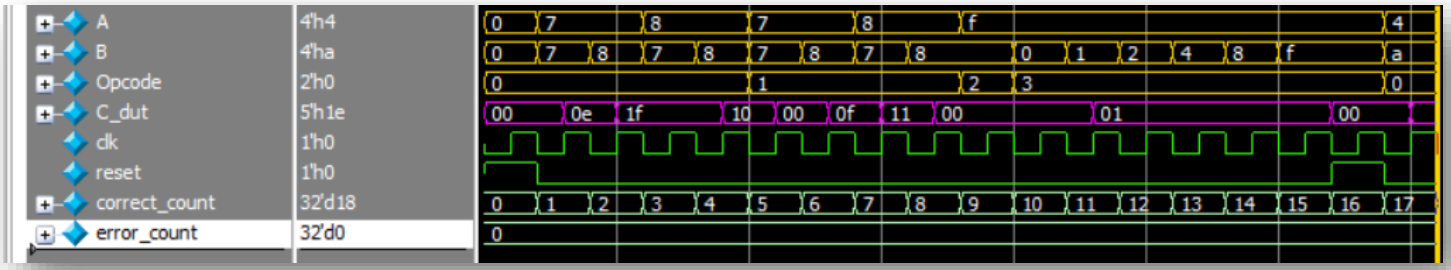


Branch Coverage:



Toggle Coverage:

6. Questasim Waveform:



7. Coverage Report:

Branch Coverage:

```
================================================================================
=== Instance: /\ALU_4_bit_tb#dut
=== Design Unit: work.ALU_4_bit
================================================================================
Branch Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Branches                         6         6         0   100.00%

================================Branch Details================================

Branch Coverage for instance /\ALU_4_bit_tb#dut

    Line         Item                      Count     Source
    ----         ----                      -----     ------
  File ALU.v
------------------------------------CASE Branch------------------------------------
    21                                       17     Count coming in to CASE
    22              1                         6               Add:            Alu_out = A + B;

    23              1                         4               Sub:            Alu_out = A - B;

    24              1                         1               Not_A:          Alu_out = ~A;

    25              1                         6               ReductionOR_B:  Alu_out = |B;

Branch totals: 4 hits of 4 branches = 100.00%

------------------------------------IF Branch------------------------------------
    32                                       16     Count coming in to IF
    32              1                         4           if (reset)

    34              1                        12           else

Branch totals: 2 hits of 2 branches = 100.00%
```

Statement Coverage:

```
Statement Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Statements                       8         8         0   100.00%
```

Toggle & Total Coverage:

```
Toggle Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Toggles                         44        44         0   100.00%

================================Toggle Details================================

Toggle Coverage for instance /\ALU_4_bit_tb#dut  --

                                          Node      1H->0L      0L->1H  "Coverage"
                                          ----------------------------------------
                                        A[0-3]           1           1      100.00
                                    Alu_out[4-0]         1           1      100.00
                                        B[0-3]           1           1      100.00
                                        C[4-0]           1           1      100.00
                                     Opcode[0-1]         1           1      100.00
                                           clk           1           1      100.00
                                         reset           1           1      100.00

Total Node Count     =         22
Toggled Node Count   =         22
Untoggled Node Count =          0

Toggle Coverage      =     100.00% (44 of 44 bins)


Total Coverage By Instance (filtered view): 100.00%
```

Q4)

1. Design:

```
module DSP(A, B, C, D, clk, rst_n, P);
parameter OPERATION = "ADD";
input  [17:0] A, B, D;
input  [47:0] C;
input clk, rst_n;
output reg  [47:0] P;

reg  [17:0] A_reg_stg1, A_reg_stg2, B_reg, D_reg, adder_out_stg1; //adder_out_stg2; Fixed Bug: Removed unused Net
reg  [47:0] C_reg, mult_out;

always @(posedge clk or negedge rst_n) begin
    if (~rst_n) begin       //Fixed bug: Used ~ instead of conditional not !
        // reset
        A_reg_stg1 <= 0;
        A_reg_stg2 <= 0;
        B_reg <= 0;
        D_reg <= 0;
        C_reg <= 0;
        adder_out_stg1 <= 0;
        mult_out <= 0;
        P <= 0;
    end
    else begin
        A_reg_stg1 <= A;
        A_reg_stg2 <= A_reg_stg1;
        B_reg <= B;
        C_reg <= C;
        D_reg <= D;
        // adder_out_stg2 <= adder_out_stg1;   Fixed Bug: Removed unused Net
        if (OPERATION == "ADD") begin
            adder_out_stg1 <= D_reg + B_reg;
            P <= mult_out + C_reg;
        end
        else if (OPERATION == "SUBTRACT") begin
            adder_out_stg1 <= D_reg - B_reg;
            P <= mult_out - C_reg;
        end
        mult_out <= A_reg_stg2 * adder_out_stg1; //Fixed Bug: Replaced adder_out_stg2 with the correct adder_out_stg1
    end
end

endmodule
```

2. Verification Plan:

Label	Design Requirment Description	Stimulus Generation	Funcational Coverage	Funcational Checking
DSP_1	When the reset is asserted, the output P value must be low	Directed during the start of simulation		A checker in the testbench to make sure the output is correct
DSP_2	Checking Addtion Operation	Randomized		A Golden Model in the testbench to make sure the output is correct

3. Test Bench:

```
module DSP_tb ();

parameter OPERATION = "ADD";
logic  [17:0] A, B, D;
logic  [47:0] C;
logic  clk, rst_n;
wire   [47:0] P_dut;
wire  [47:0] P_golden;

int correct_count, error_count;

DSP       #(OPERATION) dut    (A, B, C, D, clk, rst_n, P_dut);
DSP48A1_S #(OPERATION) golden (A, B, C, D, clk, rst_n, P_golden);

initial begin
    clk = 0;
    forever
        #1 clk = ~clk;
end

task reset_assert();
    rst_n = 0;
    check_result();
    rst_n = 1;
endtask

task check_result();
        repeat(4)@(negedge clk);
        $display("Expected:%d | DUT:%d", P_golden, P_dut);
        if(P_dut !== P_golden) begin
            error_count++;
            $stop;
        end
        else begin
            correct_count++;
        end
endtask

int i;

initial begin
    A = 0; B = 0; C = 0; D = 0; //P_golden = 0;

    correct_count = 0;
    error_count = 0;
    //DSP_1
    reset_assert();
    //DSP_2
    for(i = 0; i < 8; i++)begin
        A = $urandom; B = $urandom; C = $urandom; D = $urandom;
        check_result();

    end

    //Additional Test Vectors for 100% Code Coverage
    reset_assert();

    $display("Total Tests:%d | Correct Tests:%d | Error Test:%d", correct_count + error_count, correct_count, error_count);
    $stop;
end


endmodule

```

Golden Model:

```
1
2  module DSP48A1_S(
3      input [17:0]      A,
4      input [17:0]      B,
5      input [47:0]      C,
6      input [17:0]      D,
7      input             clk,
8      input             rst_n,
9      output reg [47:0] P
10 );
11
12 parameter OPERATION = "ADD";  //ADD or SUBTRACT
13
14 wire [17:0] A_reg;
15 wire [17:0] adder1_out;
16 wire [47:0] multiplier_out, adder2_out;
17
18
19 D_FF #(.WIDTH(18)) A_FF (A, rst_n, clk, A_reg);
20 registered_input_Adder #(.WIDTH_IN1(18), .WIDTH_IN2(18)) ADDER_1 (D, B, rst_n, clk, adder1_out);
21 registered_input_MUL #(.WIDTH_IN1(18), .WIDTH_IN2(18)) MUL (adder1_out, A_reg, rst_n, clk, multiplier_out);
22 registered_input_Adder #(.WIDTH_IN1(36), .WIDTH_IN2(48)) ADDER_2 (multiplier_out, C, rst_n, clk, adder2_out);
23
24
25 always @(posedge clk) begin
26
27     if(~rst_n)
28         P <= 0;
29     else
30         P <= adder2_out;
31 end
32
33 endmodule
34
35 module registered_input_MUL(in1, in2, rst_n, clk, out);
36
37     parameter WIDTH_IN1 = 18;
38     parameter WIDTH_IN2 = 18;
39     localparam WIDTH_OUT = WIDTH_IN1 + WIDTH_IN2;
40
41     input clk, rst_n;
42     input [WIDTH_IN1-1:0] in1;
43     input [WIDTH_IN2-1:0] in2;
44     output reg [WIDTH_OUT-1:0] out;
45
46     reg [WIDTH_IN1-1:0] in1_reg;
47     reg [WIDTH_IN2-1:0] in2_reg;
48
49     //D_FF for inputs
50     always @(posedge clk) begin
51
52         if(~rst_n)begin
53             in1_reg <= 0;
54             in2_reg <= 0;
55         end
56         else begin
57             in1_reg <= in1;
58             in2_reg <= in2;
59         end
60
61     end
62
63     //Combinational output
64     always @(*) begin
65
66         out = in1_reg * in2_reg;
67     end
68 endmodule
```

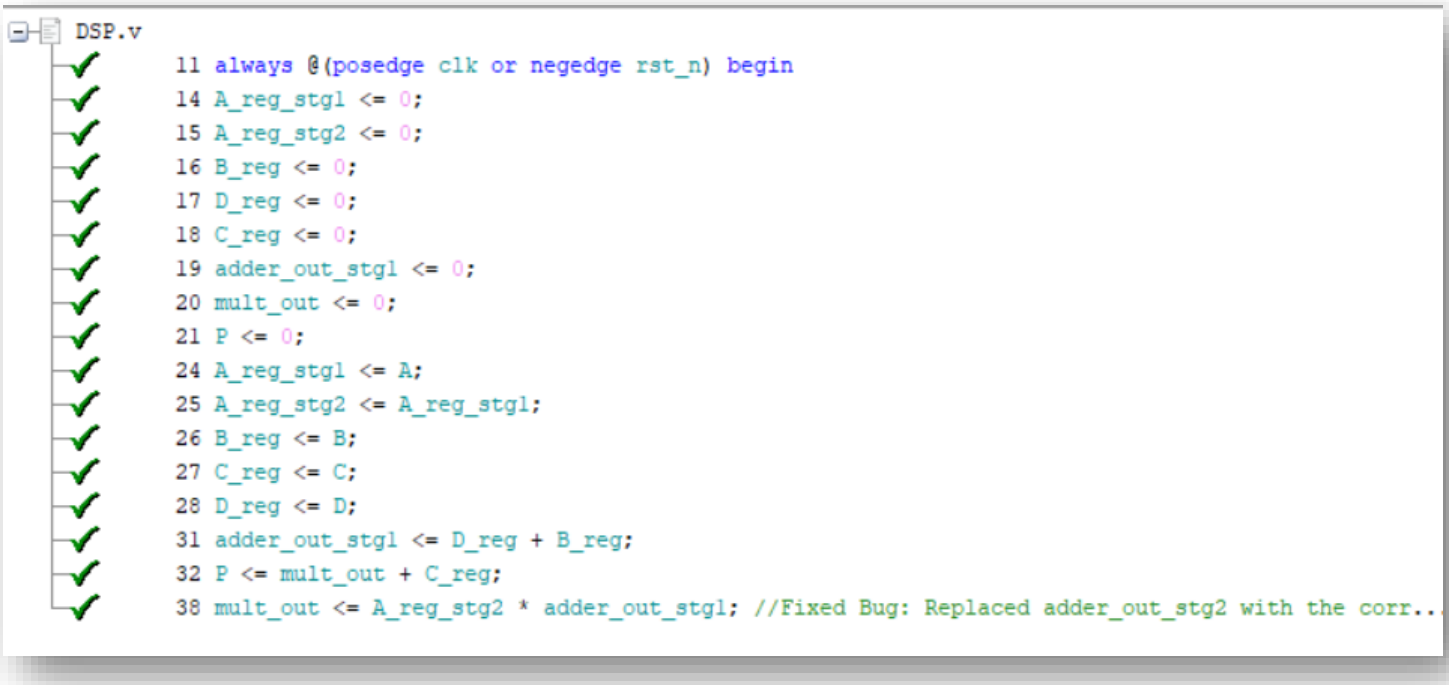
```
module registered_input_Adder(in1, in2, rst_n, clk, out);

    parameter WIDTH_IN1 = 18;
    parameter WIDTH_IN2 = 18;
    localparam WIDTH_OUT = (WIDTH_IN1 >= WIDTH_IN2 ) ? WIDTH_IN1: WIDTH_IN2;
    parameter OPERATION = "ADD";
    input clk, rst_n;
    input [WIDTH_IN1-1:0] in1;
    input [WIDTH_IN2-1:0] in2;
    output reg [WIDTH_OUT-1:0] out;

    reg [WIDTH_IN1-1:0] in1_reg;
    reg [WIDTH_IN2-1:0] in2_reg;

    //D_FF for inputs
    always @(posedge clk) begin

        if(~rst_n)begin
            in1_reg <= 0;
            in2_reg <= 0;
        end
        else begin
            in1_reg <= in1;
            in2_reg <= in2;
        end

    end

    //Combinational output
    always @(*) begin

        if(OPERATION == "ADD")
            out = in1_reg + in2_reg;
        else if(OPERATION == "SUBTRACT")
            out = in1_reg - in2_reg;

    end
endmodule

```

4. Do file:

```
vlib work
vlog DSP.v DSP_tb.sv DSP_Golden.v D_FF.v  +cover -covercells
vsim -voptargs=+acc work.DSP_tb -cover
add wave *
coverage save DSP.ucdb -onexit -du work.DSP
run -all
#quit -sim
#vcover report DSP.ucdb -details -annotate -all -output coverage_rpt.txt
```

5. Code Coverage

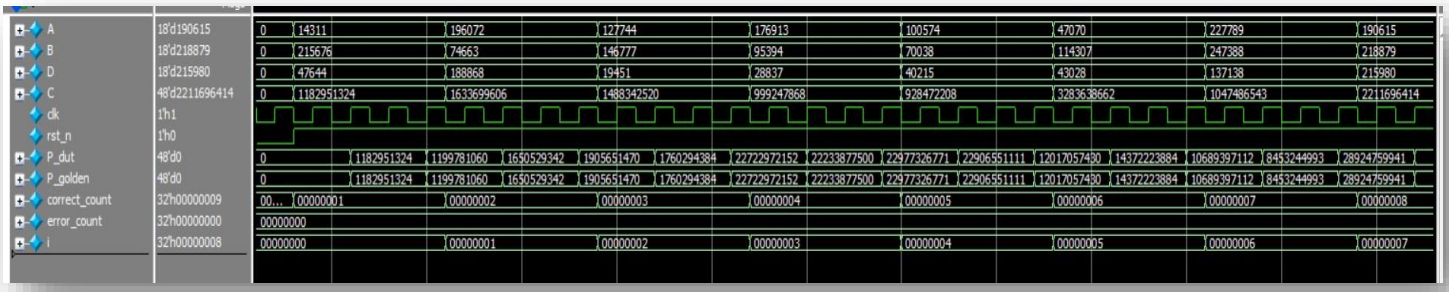
Statement Coverage:



Branch Coverage:

Toggle Coverage:

6. Questasim Waveform:



7. Coverage Report

Branch Coverage:

```
================================================================================
=== Instance: /\DSP_tb#dut
=== Design Unit: work.DSP
================================================================================
Branch Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Branches                         2         2         0   100.00%

================================Branch Details================================

Branch Coverage for instance /\DSP_tb#dut

    Line         Item                      Count     Source
    ----         ----                      -----     ------
  File DSP.v
------------------------------------IF Branch------------------------------------
    12                                       132     Count coming in to IF
    12              1                          4        if (~rst_n) begin              //Fixed bug: Use

    23              1                        128        else begin

Branch totals: 2 hits of 2 branches = 100.00%
```

Statement Coverage:

```
Statement Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Statements                      17        17         0   100.00%

================================Statement Details================================
```

Toggle & Total Coverage:

```
Toggle Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Toggles                        676       564       112    83.43%

================================Toggle Details================================

Toggle Coverage for instance /\DSP_tb#dut  --

                                            Node      1H->0L      0L->1H  "Coverage"
                                            ----------------------------------------
                                      A[0-17]           1           1      100.00
                                 A_reg_stg1[17-0]       1           1      100.00
                                 A_reg_stg2[17-0]       1           1      100.00
                                      B[0-17]           1           1      100.00
                                   B_reg[17-0]          1           1      100.00
                                      C[0-31]           1           1      100.00
                                     C[32-47]           0           0        0.00
                                  C_reg[47-32]          0           0        0.00
                                   C_reg[31-0]          1           1      100.00
                                      D[0-17]           1           1      100.00
                                   D_reg[17-0]          1           1      100.00
                                     P[47-36]           0           0        0.00
                                     P[35-0]            1           1      100.00
                           adder_out_stg1[17-0]         1           1      100.00
                                         clk            1           1      100.00
                                mult_out[47-36]         0           0        0.00
                                 mult_out[35-0]         1           1      100.00
                                      rst_n             1           1      100.00

Total Node Count     =        338
Toggled Node Count   =        282
Untoggled Node Count =         56

Toggle Coverage      =      83.43% (564 of 676 bins)


Total Coverage By Instance (filtered view): 94.47%
```

Note: 100% total Coverage was not achieved due to some bits in input C not getting a value for $random function.